

6. Heap Sort

Comparison-based sorting technique based on Binary Heap Data Structures. In Heap Sort, we use Binary Heap so that we can quickly find and move the max element in $O(\log n)$ instead of $O(n)$ and hence achieve the $O(n \log n)$ time complexity.

First convert the array into a max heap using heapify. The array elements need to be changed to follow heap properties. Then one by one delete the root node of the Max-heap and replace it with the last node and heapify. Repeat this process while the size of heap is greater than 1.

- Rearrange array elements so that they form a Max Heap.
- Repeat the following steps until the heap contains only one element.
 - Swap the root element of the heap (which is the largest element in current heap) with the last element of the heap.
 - Remove the last element of the heap (which is now in the root position). We mainly reduce heap size and do not remove

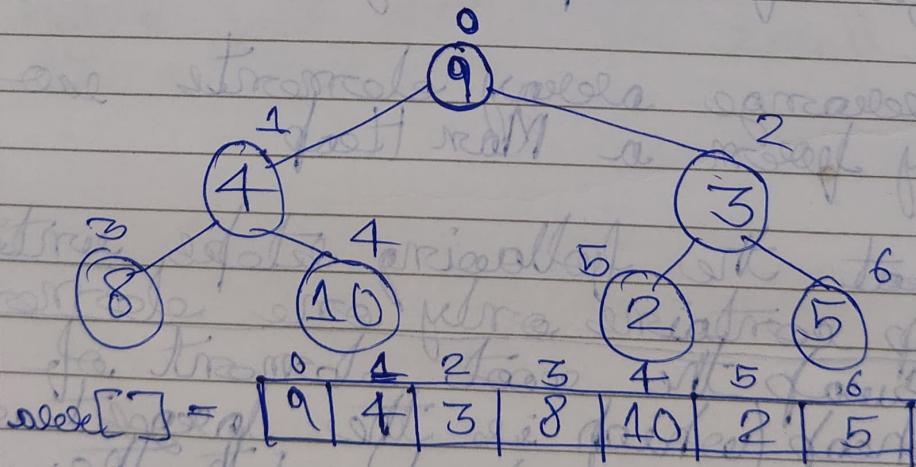
element from the actual array.
 • Heapify the remaining elements of the heap.

• Finally we get sorted array.

• Working of Heap Sort

Step 1: Treat the Array as a Complete Binary Tree

We first need to visualize the array as a complete binary tree. For an array of size n , the root is at index 0, the left child of an element at index i is at $2i+1$, and the right child is at $2i+2$.

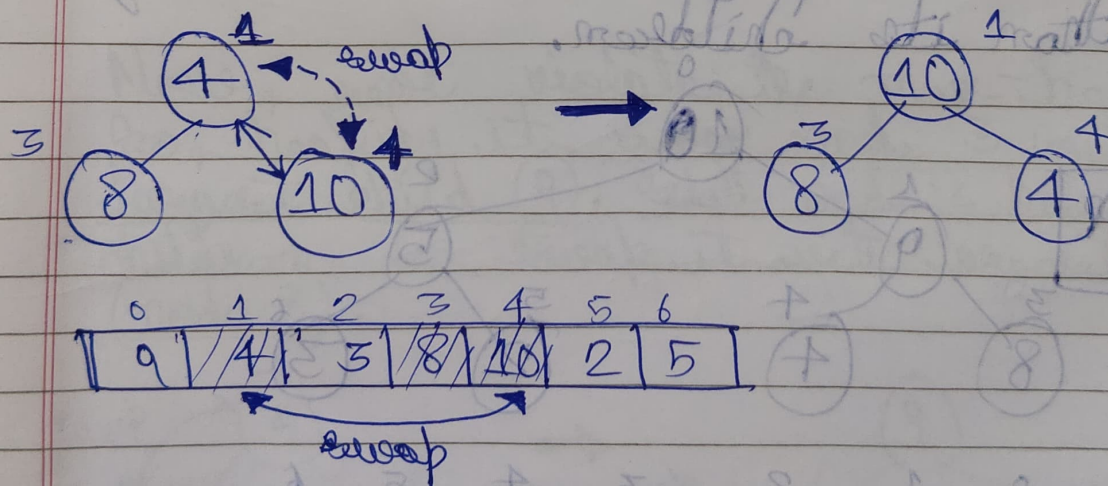


Step 2: Build a Max Heap

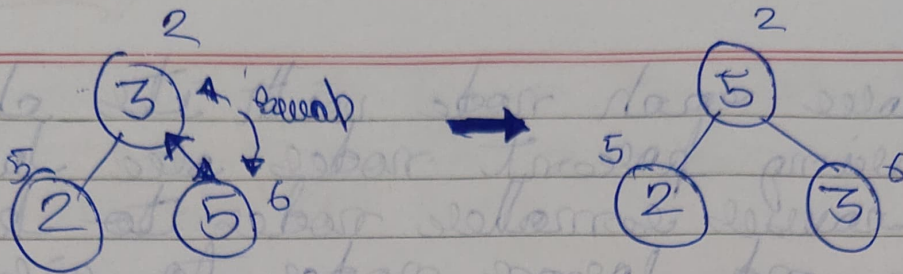
Compare each node with its children, ensuring parent nodes are larger. This causes smaller nodes to bubble down and larger nodes to rise to the top. Let's begin with leaf nodes (elements at the end).

Let's look in the next upper level (node 4 and 3).

Node 4 is smaller than its child node (10), so swap it with the larger child to maintain the property that the parent should be larger than its children.



Check for the next node (3) in parent level. Since, it has a larger child, so swap it to ensure the parent has a larger value.

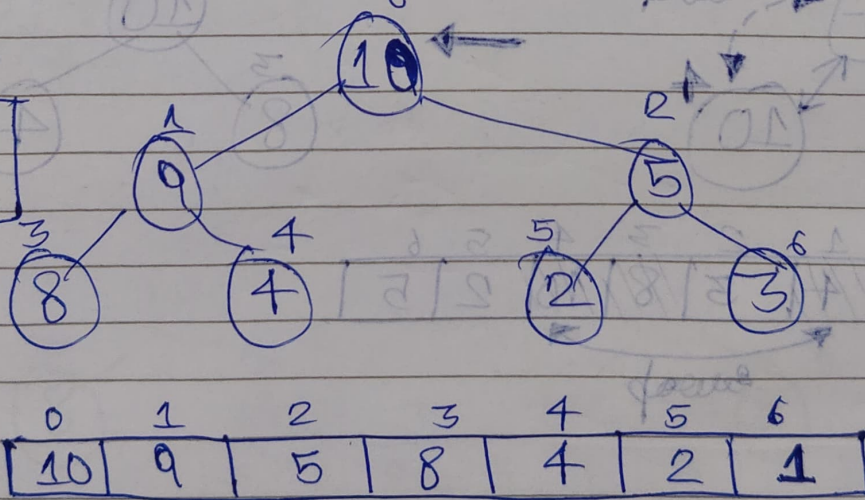


After the completion of current level, we have now two smaller valid max heap.

Let's move to the next upper level, here we're node 9 at the root.

Node 9 is smaller than its child node (10), so swap it with the larger child to maintain the property that the parent should be larger than its children.

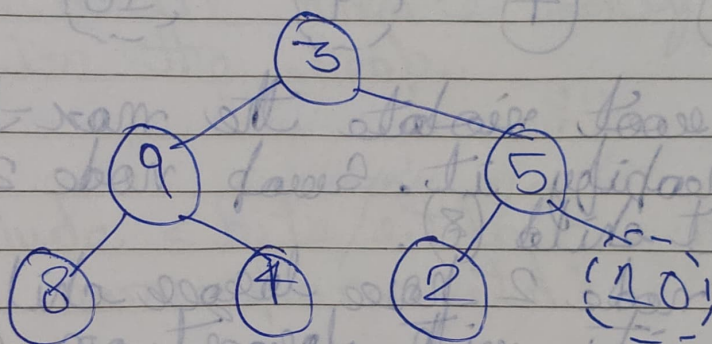
Max Heap Form



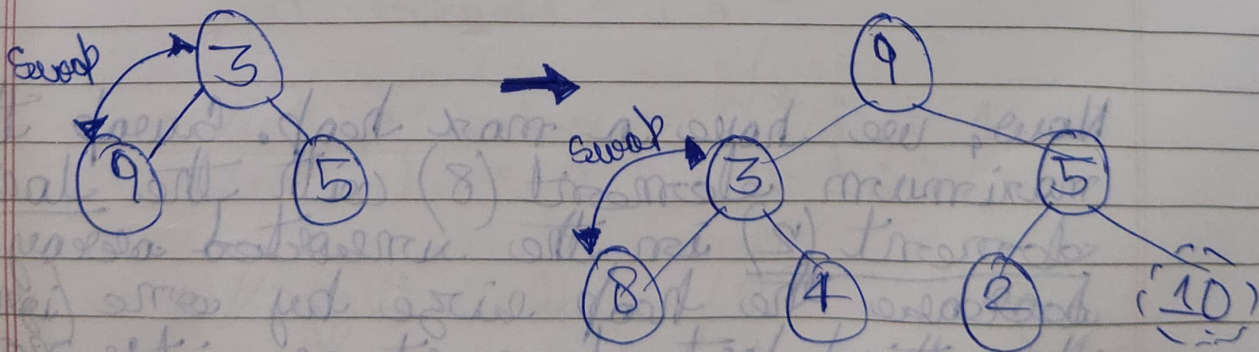
Step 3: Sort the array by placing largest element at end of unsorted array.

We've transformed the given array to follow the max heap property.

Swap the maximum element (10) with the last element (3) in the unsorted array. Decrease the size of the heap by one (ignore the last element, as it is now sorted).

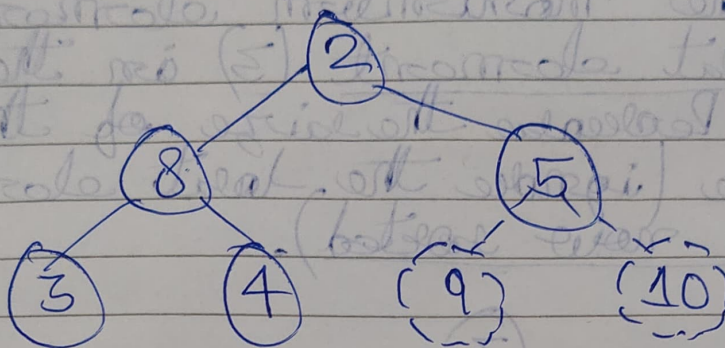


Now, root violates the max-heap property, so, heapify it. Swap node 3 with its largest child (9). Still node 3 has larger children, so swap it with largest one (node 8).

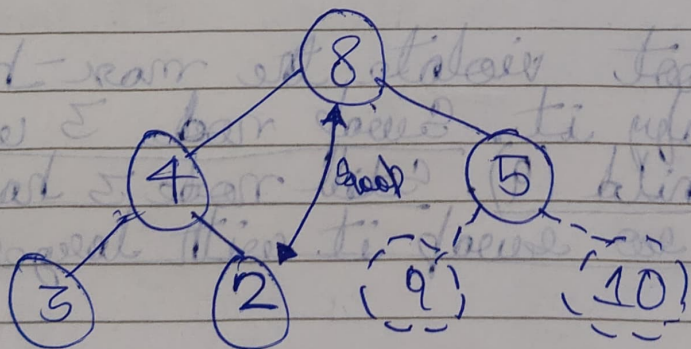


Now, we have a max heap. Swap the maximum element (9) with the last element (2) in the unsorted array, then

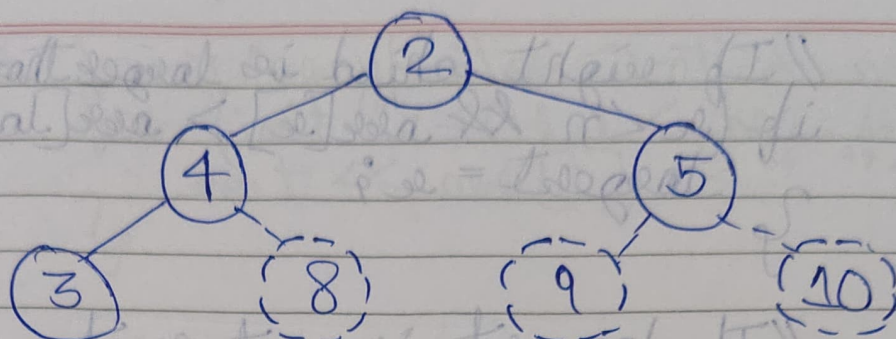
Decrease the heap size by one (ignoring the second last element as it's now sorted).



Now, root violates the max-heap property, so, heapify it. Swap node 2 with its largest child (8).
 still node 2 has larger children, so swap it with largest one (node 4).



Now, we have a max heap. Swap the maximum element (8) with the last element (2) in the unsorted array, then decrease the heap size by one (ignoring the third last element as it's now sorted).



We need to keep repeating these steps until there's only one element left in the heap.

```
#include <bits/stdc++.h>
using namespace std;
```

// To heapify a subtree rooted with node
// i which is an index in arr

```
void heapify(vector<int> &arr, int n,
             int i)
```

```
{
    // Initialize largest as root
    int largest = i;
```

```
    // left index = 2*i + 1
    int l = 2 * i + 1;
```

```
    // right index = 2*i + 2
    int r = 2 * i + 2;
```

```
    // If left child is larger than root
    if (l < n & arr[l] > arr[largest])
        largest = l;
```

```
// If right child is larger than largest so far
if (e < n && arr[e] > arr[largest]) {
    largest = e;
}
```

```
// If largest is not root
if (largest != i) {
    swap(arr[i], arr[largest]);
}
```

```
// Recursively heapify the affected sub-tree
heapify(arr, n, largest);
}
```

```
void heapSort(vector<int> &arr) {
    int n = arr.size();
```

```
// Build heap (rearrange array)
for (int i = n/2 - 1; i >= 0; i--) {
    heapify(arr, n, i);
}
```

```
// One by one extract an element from heap
for (int i = n - 1; i > 0; i--) {
```

```
    // Move current root to end
    swap(arr[0], arr[i]);
```

```
    // Call max heapify on the reduced heap
    heapify(arr, i, 0);
}
```

```

void printArray(vector<int> & arr) {
    for (int i = 0; i < arr.size(); i++) {
        cout << arr[i] << " ";
    }
}

```

```

int main() {

```

```

    vector<int> arr = {9, 4, 3, 8, 10, 2, 5};

```

```

    // Function call
    heapSort(arr);

```

```

    cout << "Sorted array: ";
    printArray(arr);
}

```

```

// Sorted array: 2 3 4 5 8 9 10

```

- Time Complexity: $O(n \log n)$
- Auxiliary Space: $O(\log n)$, due to the recursive call stack. However, auxiliary space can be $O(1)$ for iterative implementation.
- Important points about Heap Sort
 1. In-place algorithm.
 2. Its typical implementation is not stable but can be made stable.

3. Typically 2-3 times slower than well-implemented Quicksort. The reason for slowness is a lack of locality of reference.

• Advantages of Heap Sort

1. Efficient Time Complexity - $O(n \log n)$

• Disadvantages of Heap Sort

1. Costly as the constants are higher compared to Merge sort even if the time complexity is $O(n \log n)$ for both.
2. Unstable, it might exchange the relative order.
3. Not very efficient because of the high constants in the time complexity.